

Stacks and Queues

Md Shamsuzzoha Bayzid

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology



Stack: applications

Operating Systems (memory management), Web browsers, Editors,
Compilers

Recursions

Stack: Recursions

`factorial (n) = n * factorial(n-1)`

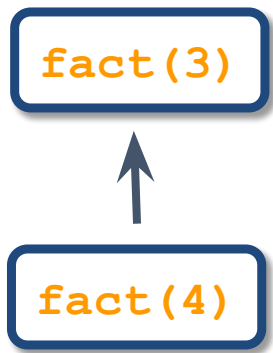
```
// Recursive function to calculate factorial  
int factorial(int n) {  
    // Base case  
    if (n == 0 || n == 1)  
        return 1;  
    // Recursive case  
    else  
        return n * factorial(n - 1);  
}
```

Stack: Recursions

```
// Recursive function to calculate factorial  
int factorial(int n) {  
    // Base case  
    if (n == 0 || n == 1)  
        return 1;  
    // Recursive case  
    else  
        return n * factorial(n - 1);  
}
```

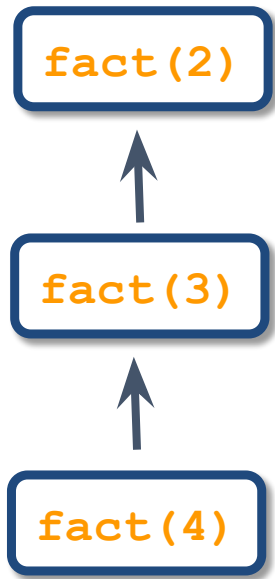
fact(4)

Stack: Recursions



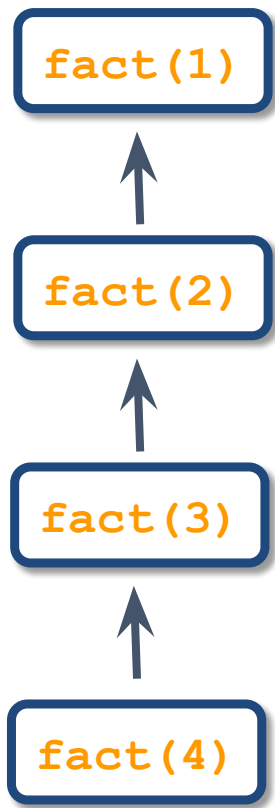
```
// Recursive function to calculate factorial  
int factorial(int n) {  
    // Base case  
    if (n == 0 || n == 1)  
        return 1;  
    // Recursive case  
    else  
        return n * factorial(n - 1);  
}
```

Stack: Recursions



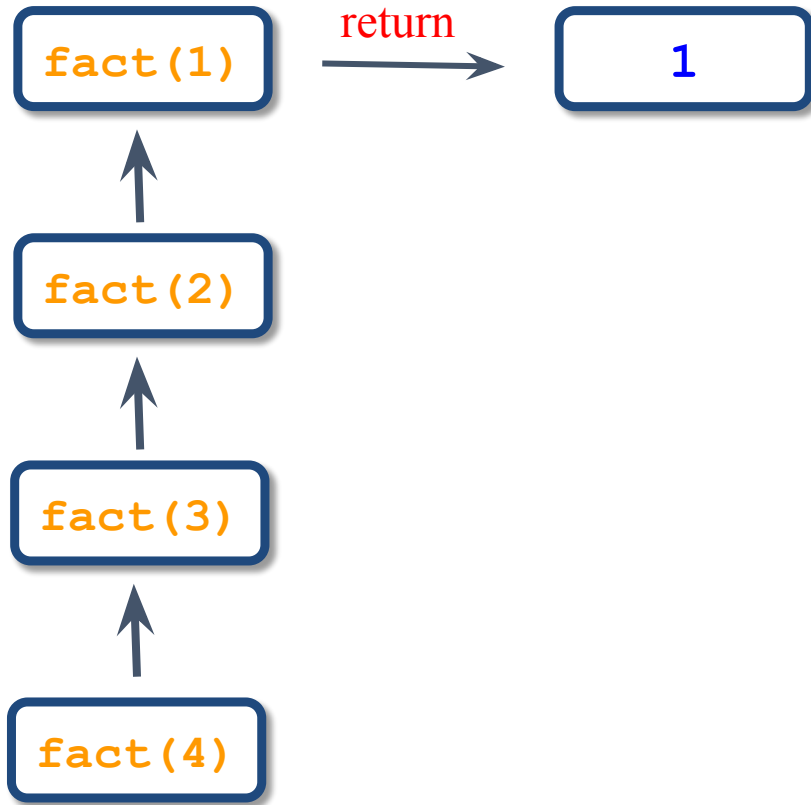
```
// Recursive function to calculate factorial  
int factorial(int n) {  
    // Base case  
    if (n == 0 || n == 1)  
        return 1;  
    // Recursive case  
    else  
        return n * factorial(n - 1);  
}
```

Stack: Recursions



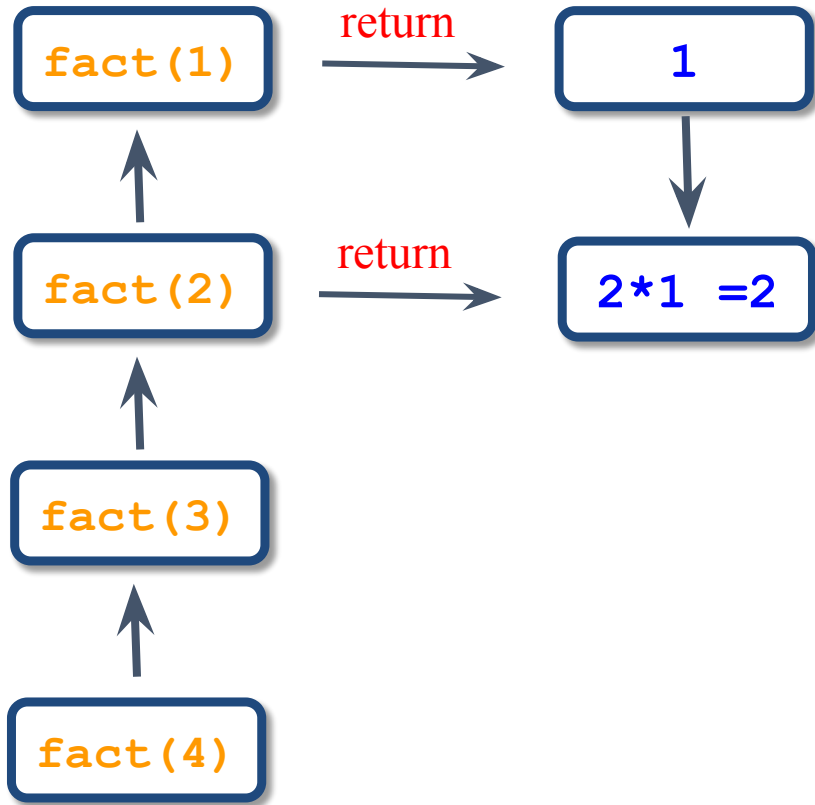
```
// Recursive function to calculate factorial  
int factorial(int n) {  
    // Base case  
    if (n == 0 || n == 1)  
        return 1;  
    // Recursive case  
    else  
        return n * factorial(n - 1);  
}
```

Stack: Recursions



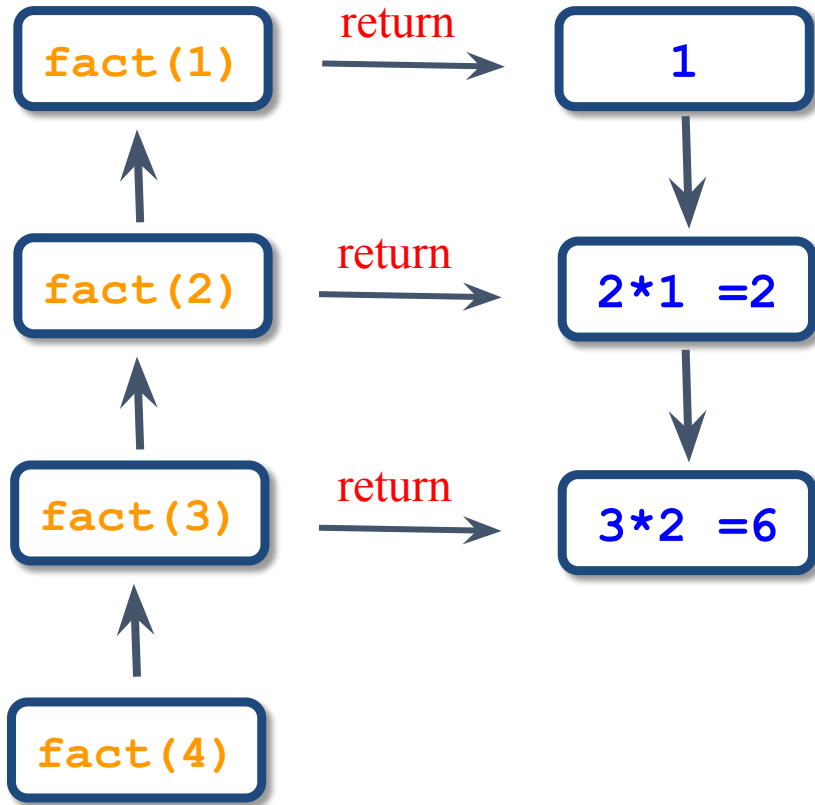
```
// Recursive function to calculate factorial  
int factorial(int n) {  
    // Base case  
    if (n == 0 || n == 1)  
        return 1;  
    // Recursive case  
    else  
        return n * factorial(n - 1);  
}
```

Stack: Recursions



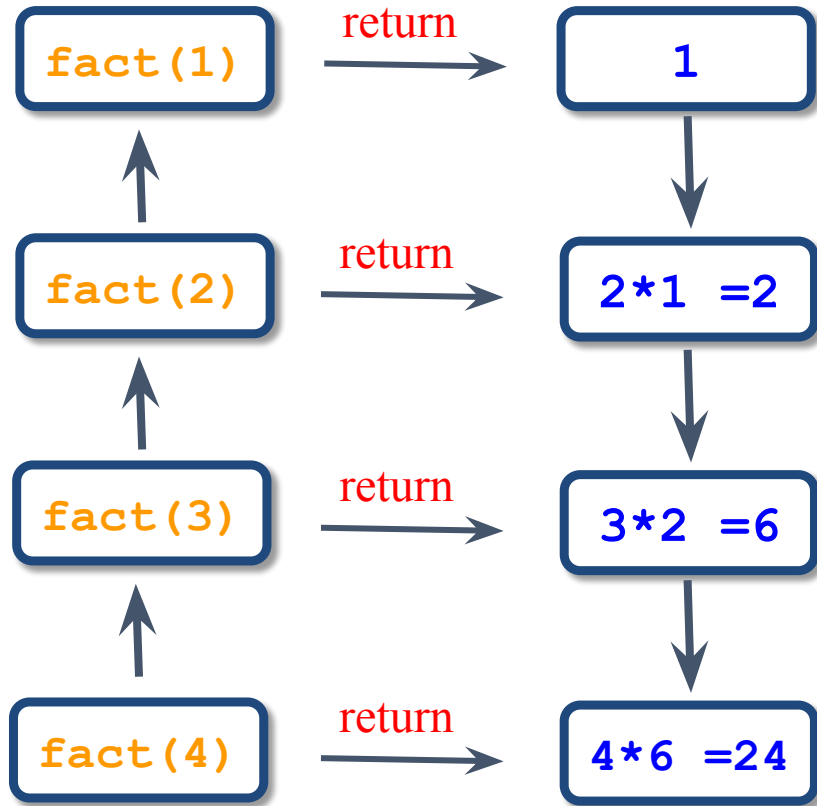
```
// Recursive function to calculate factorial  
int factorial(int n) {  
    // Base case  
    if (n == 0 || n == 1)  
        return 1;  
    // Recursive case  
    else  
        return n * factorial(n - 1);  
}
```

Stack: Recursions



```
// Recursive function to calculate factorial  
int factorial(int n) {  
    // Base case  
    if (n == 0 || n == 1)  
        return 1;  
    // Recursive case  
    else  
        return n * factorial(n - 1);  
}
```

Stack: Recursions

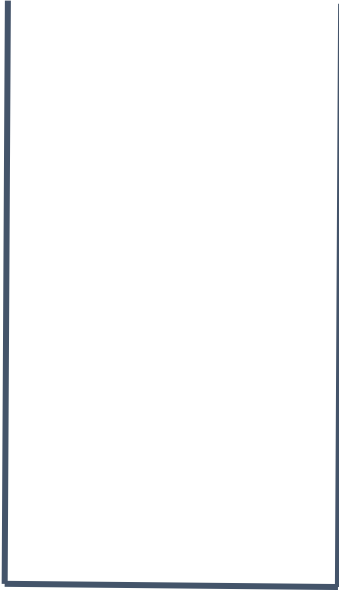


```
// Recursive function to calculate factorial  
int factorial(int n) {  
    // Base case  
    if (n == 0 || n == 1)  
        return 1;  
    // Recursive case  
    else  
        return n * factorial(n - 1);  
}
```

Stack: String reverse



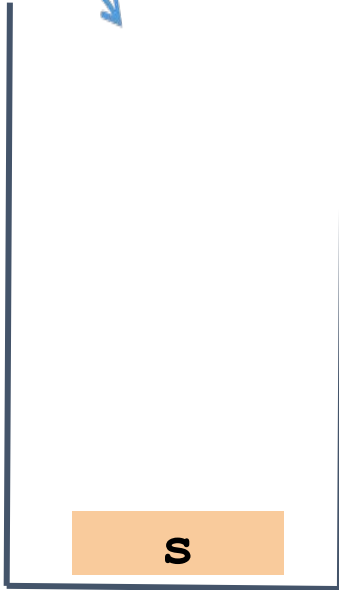
s t r e s s e d



Stack: String reverse



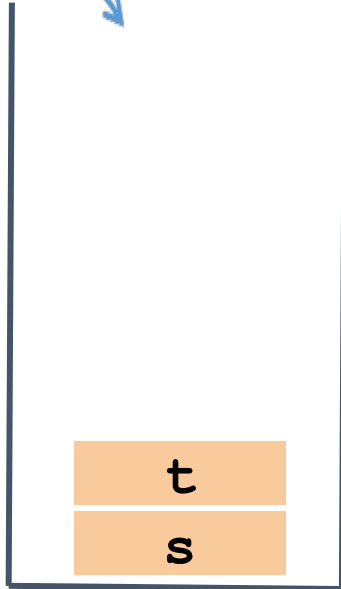
s t r e s s e d



Stack: String reverse



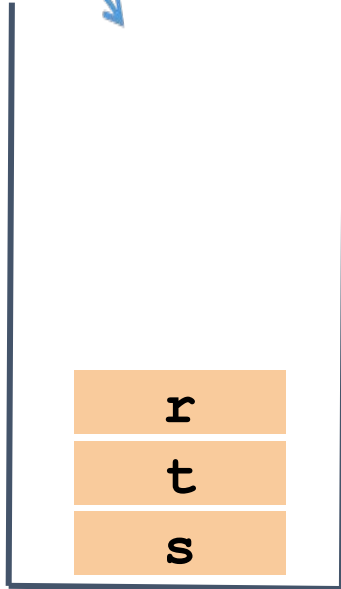
s t r e s s e d



Stack: String reverse



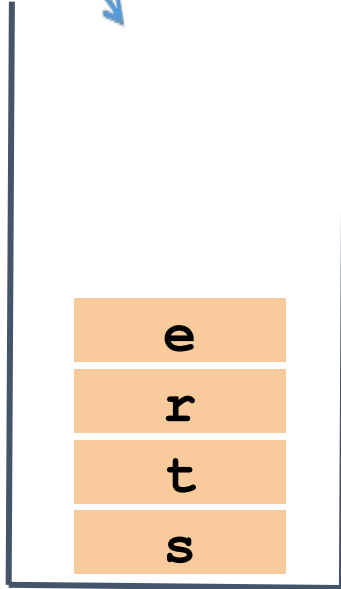
s t r e s s e d



Stack: String reverse



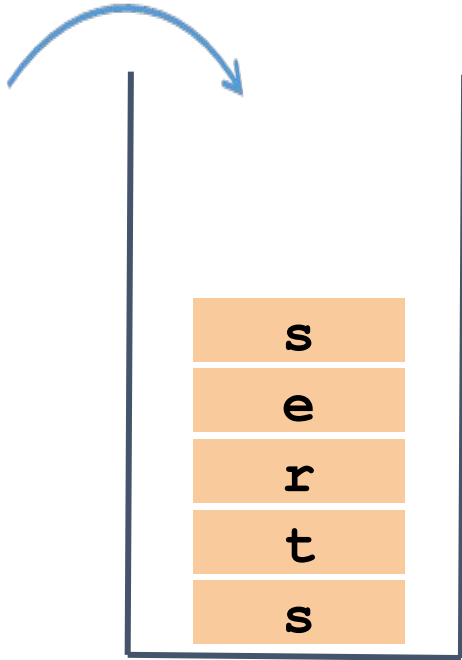
s t r e s s e d



Stack: String reverse



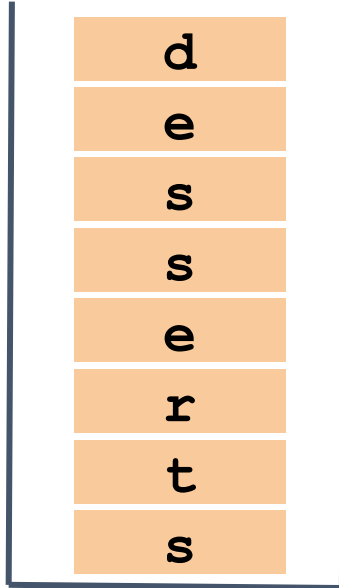
s t r e s s e d



Stack: String reverse

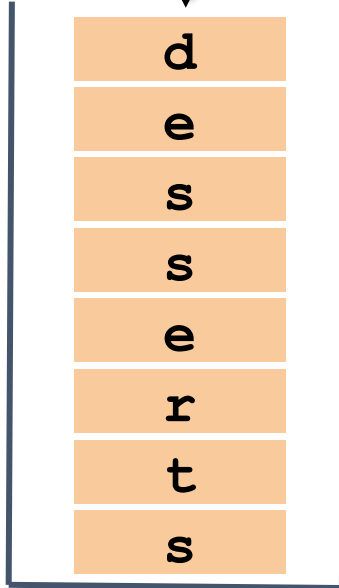


s t r e s s e d



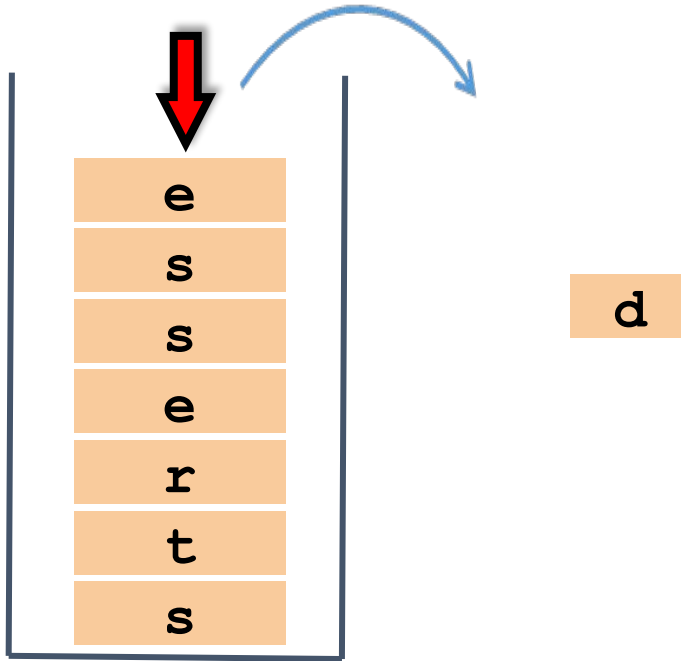
Stack: String reverse

s t r e s s e d



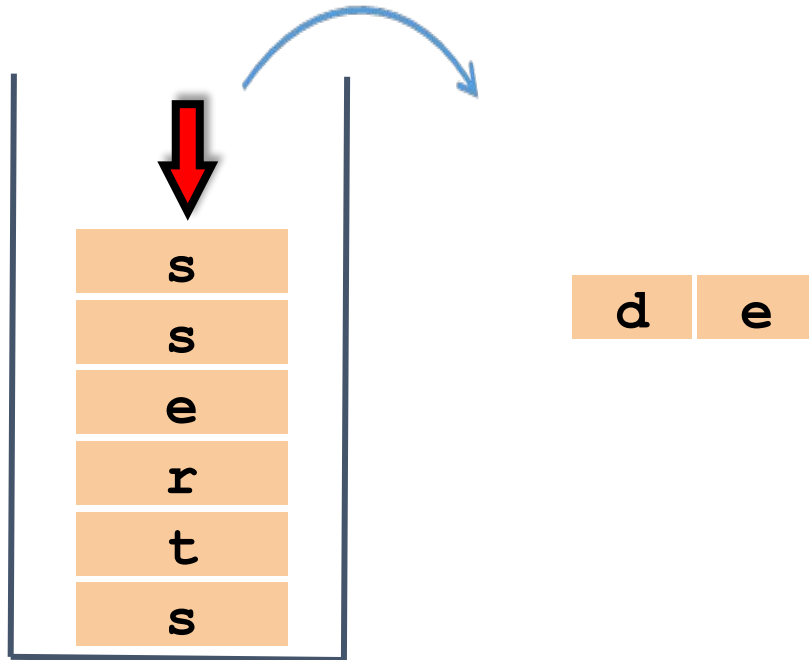
Stack: String reverse

s t r e s s e d



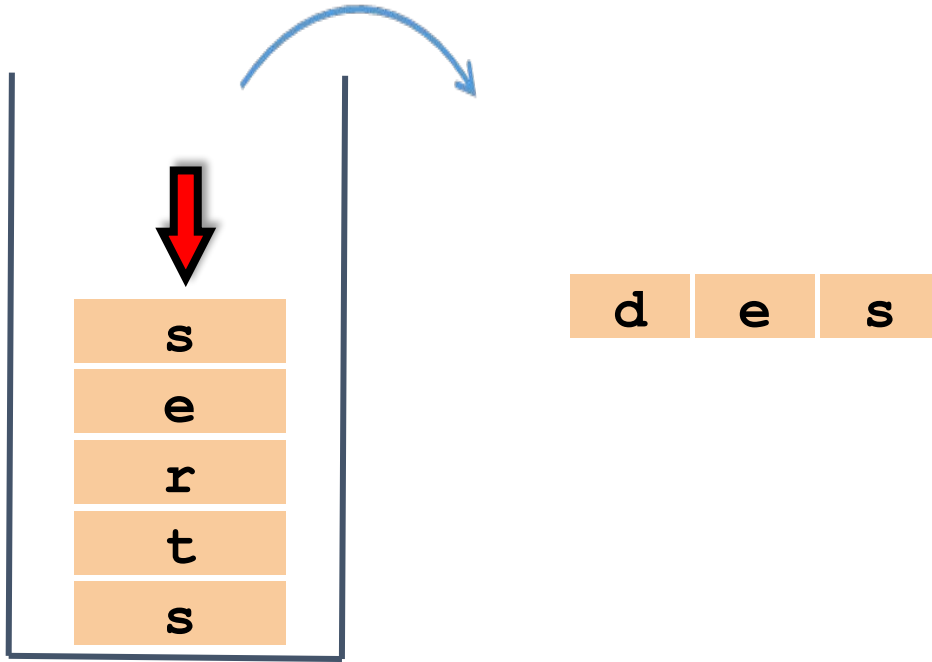
Stack: String reverse

s t r e s s e d



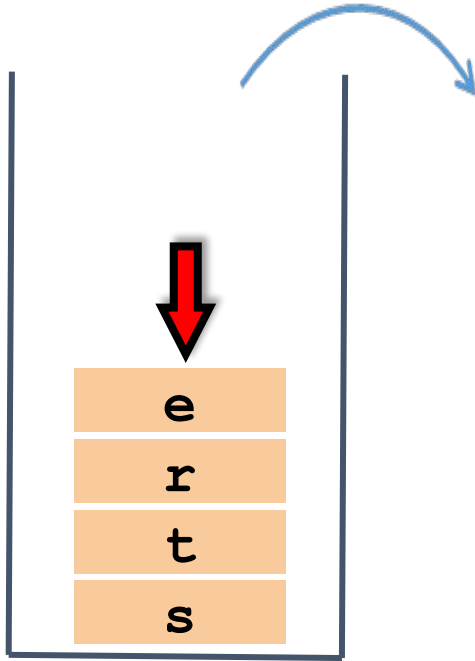
Stack: String reverse

s t r e s s e d



Stack: String reverse

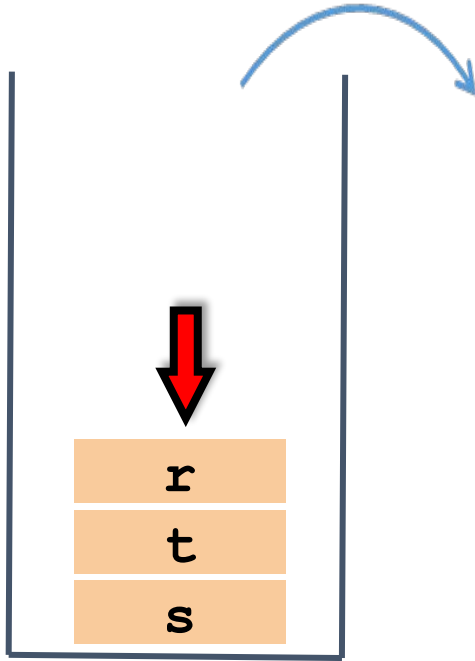
s t r e s s e d



d e s s

Stack: String reverse

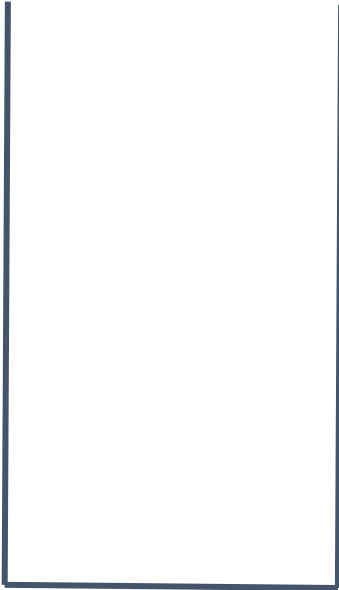
s t r e s s e d



d e s s e

Stack: String reverse

s t r e s s e d



d e s s e r t s

Stack: Balanced Parentheses

[{ (a+b) (c*d) } (e/f)]

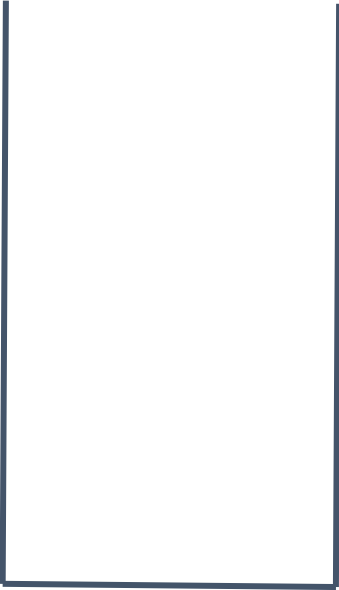
((() (())))

[() { (})]

Stack: Balanced Parentheses



[	{	(	a	+	b	)	(	c	*	d	)	}	(	e	/	f	)	]
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

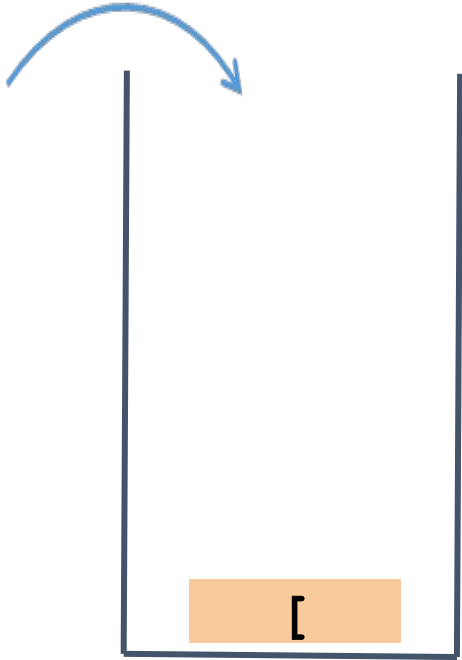


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

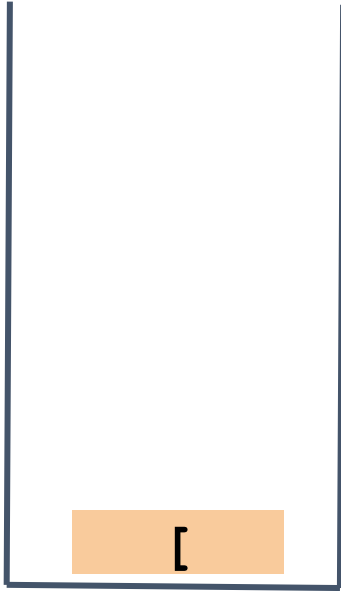


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

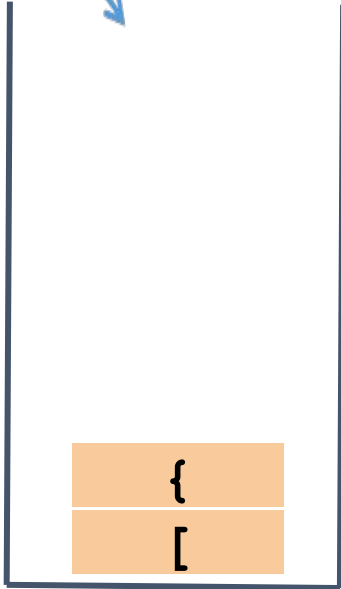


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

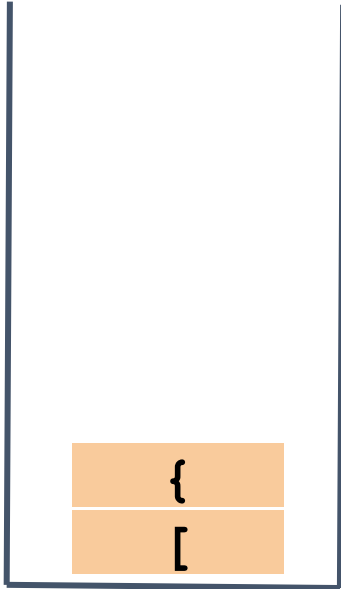


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

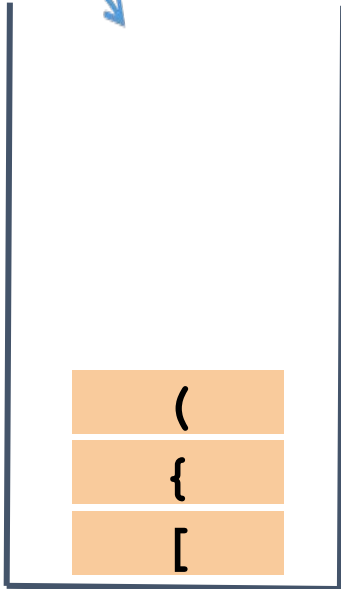


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

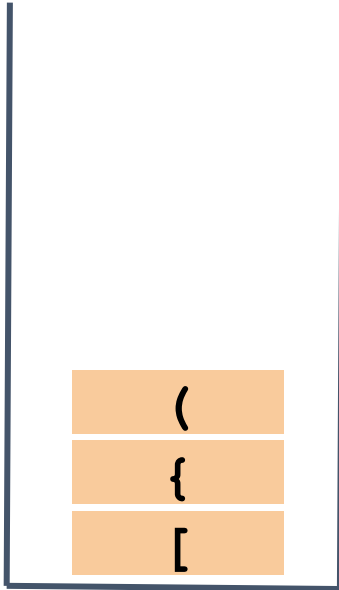


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

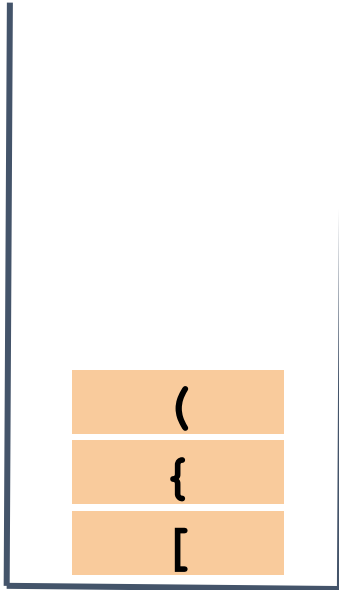


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

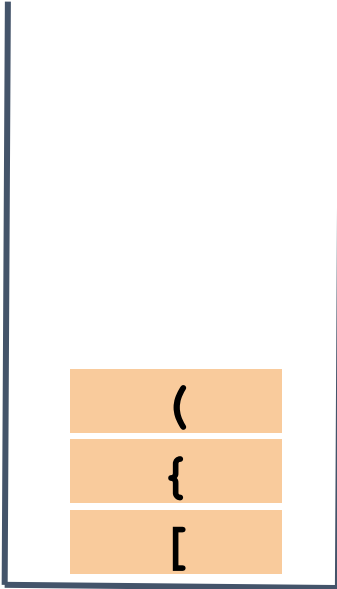


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

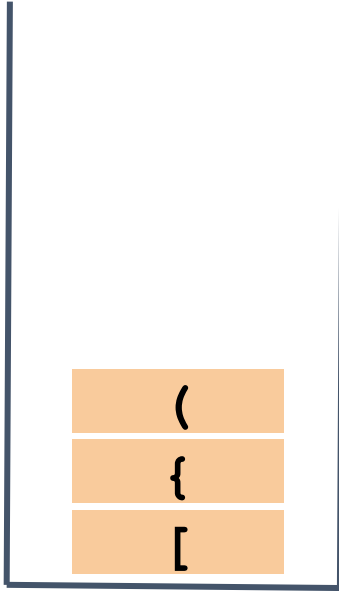


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

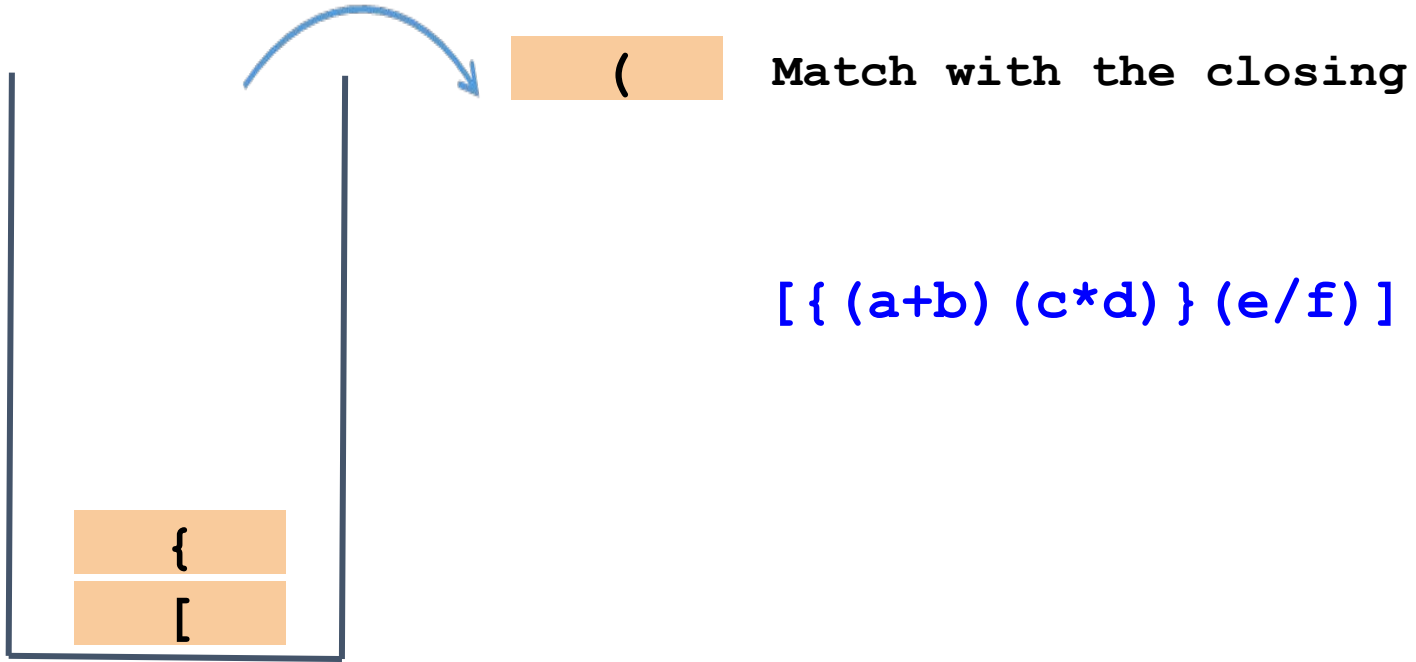


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



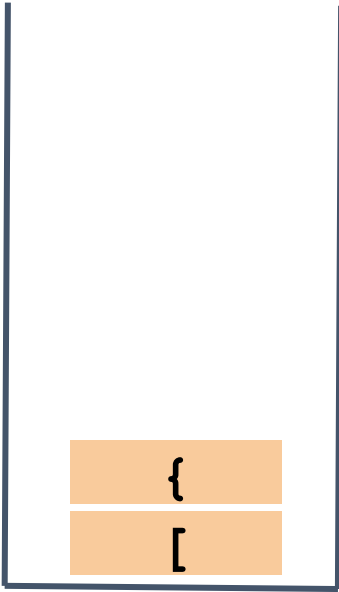
[{ (a + b) (c * d) } (e / f)]



Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

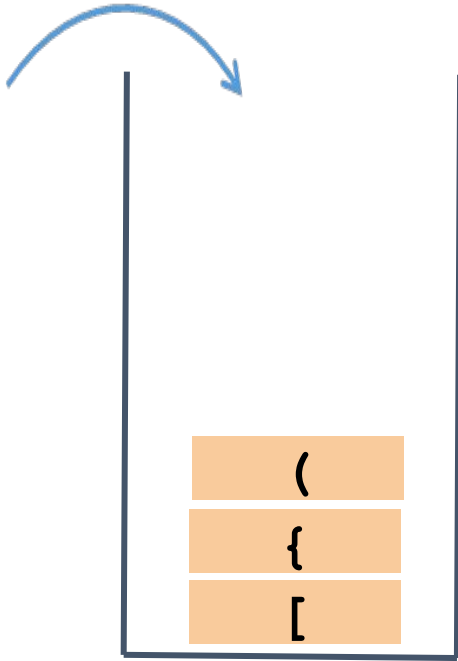


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

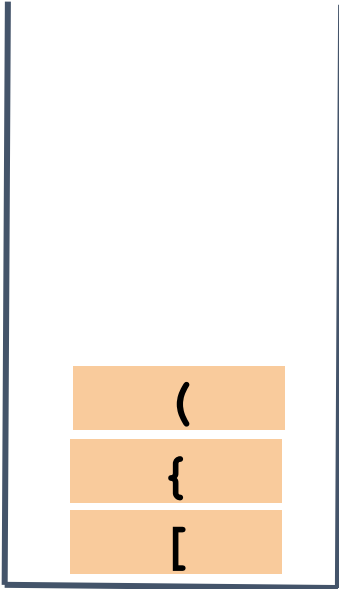


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

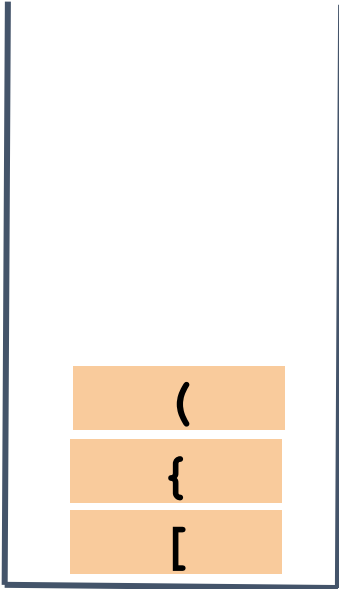


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

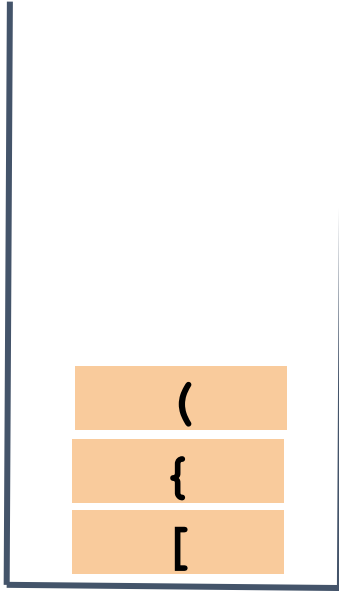


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

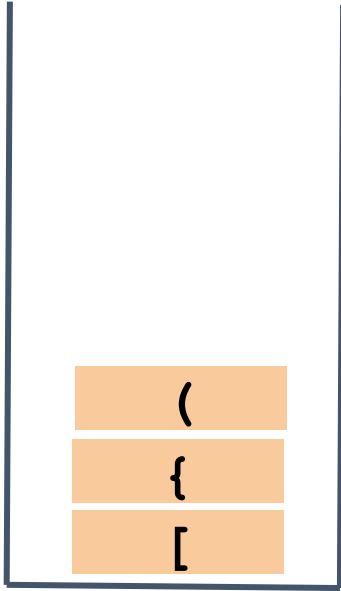


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

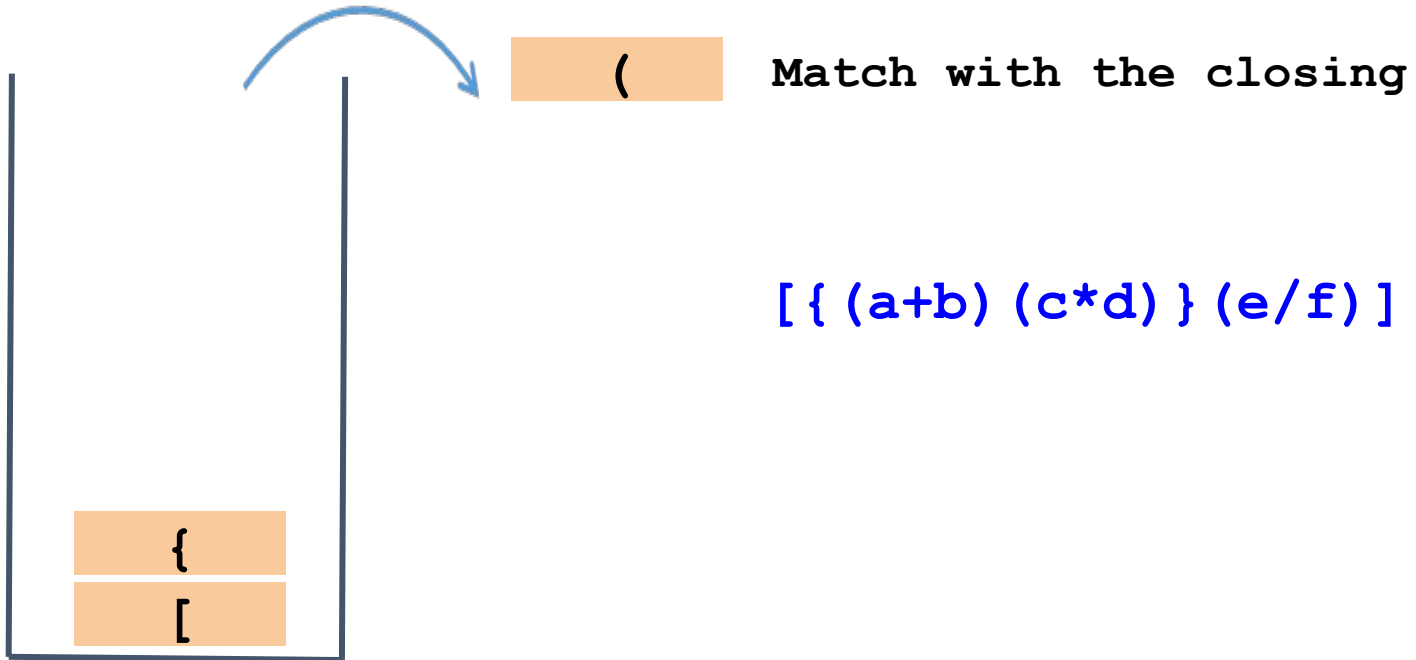


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



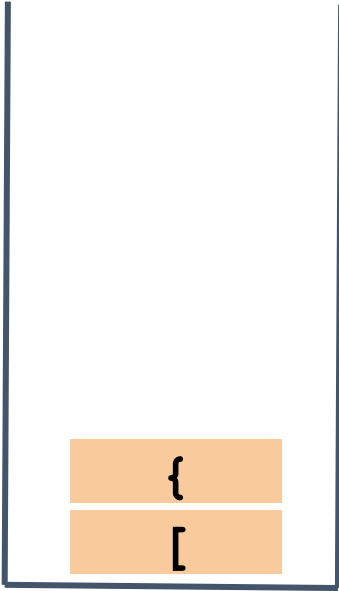
[{ (a + b) (c * d) } (e / f)]



Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

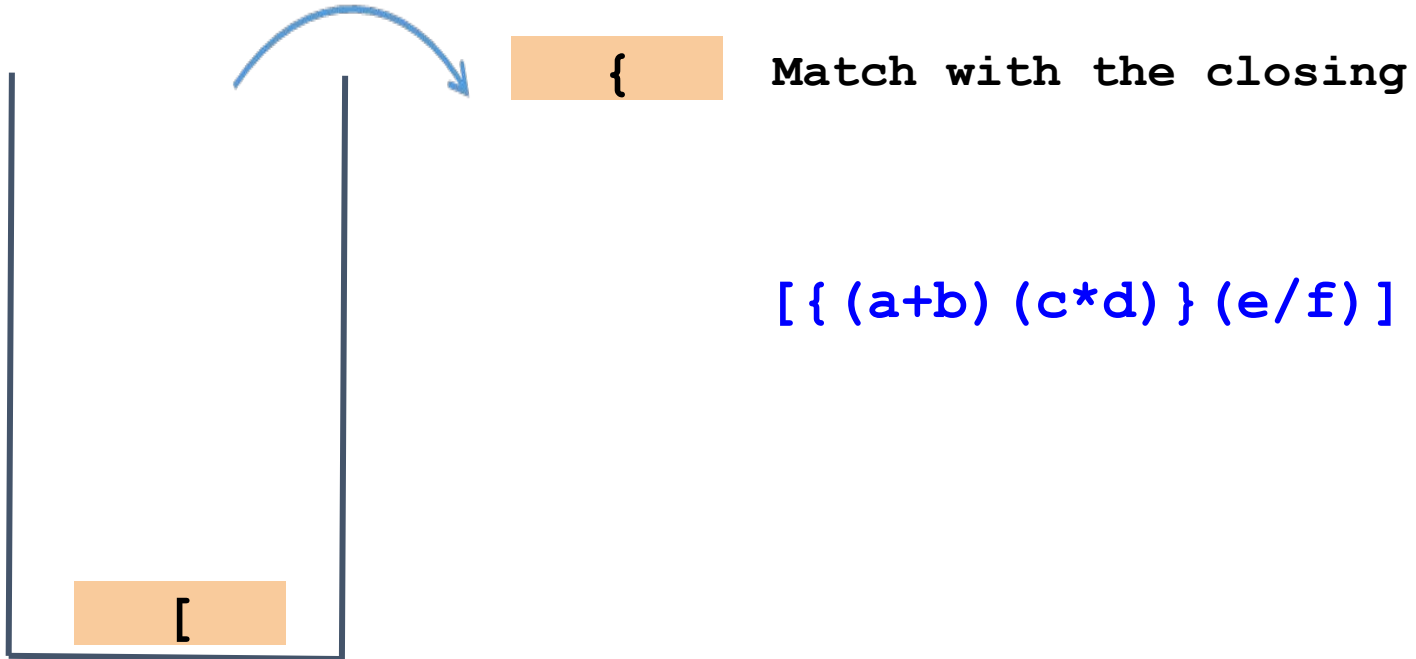


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



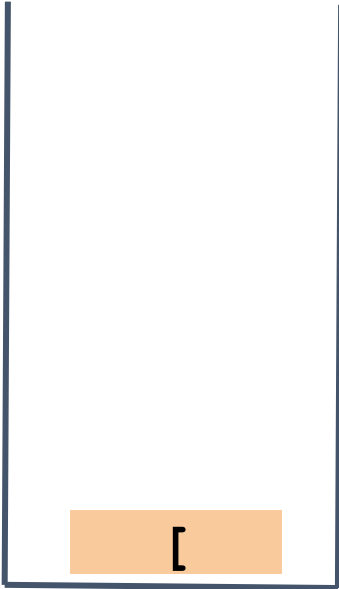
[{ (a + b) (c * d) } (e / f)]



Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

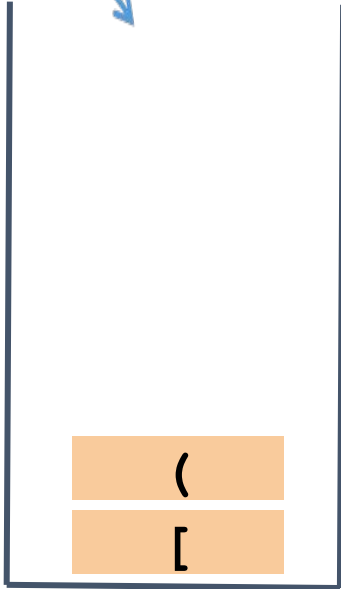


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

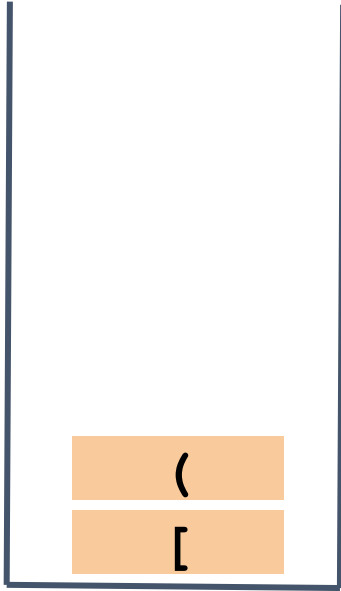


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

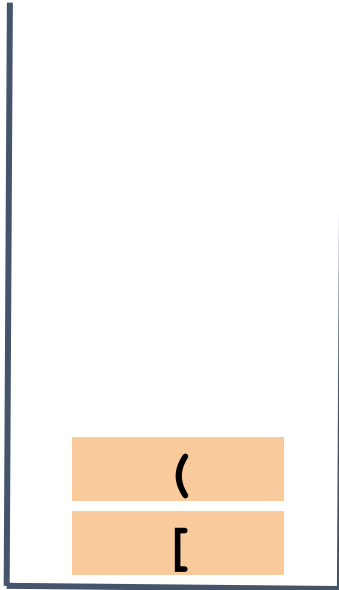


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

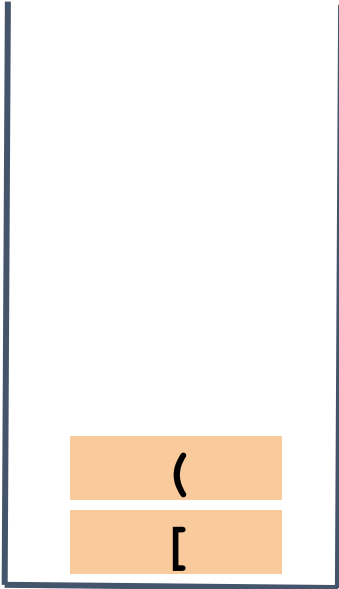


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

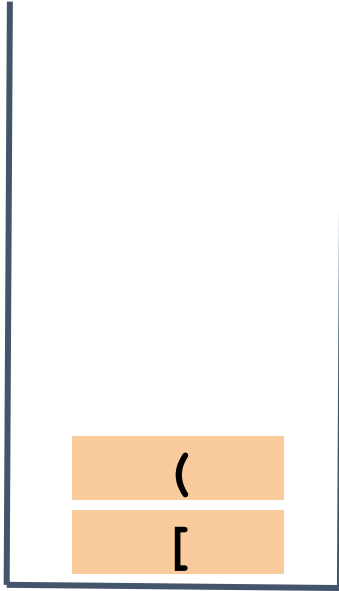


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

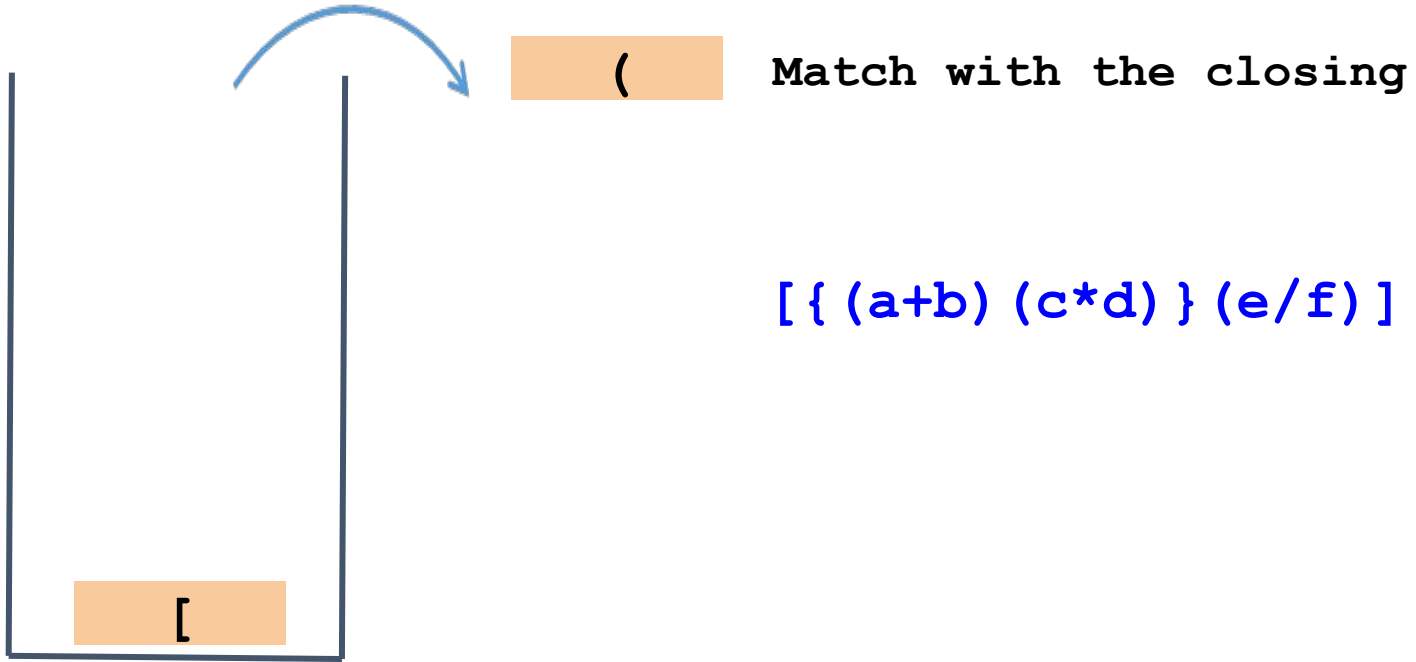


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



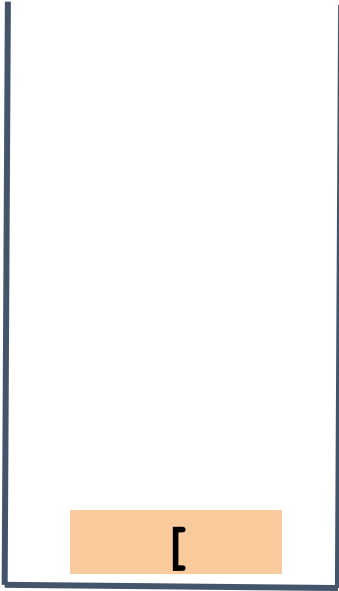
[{ (a + b) (c * d) } (e / f)]



Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]

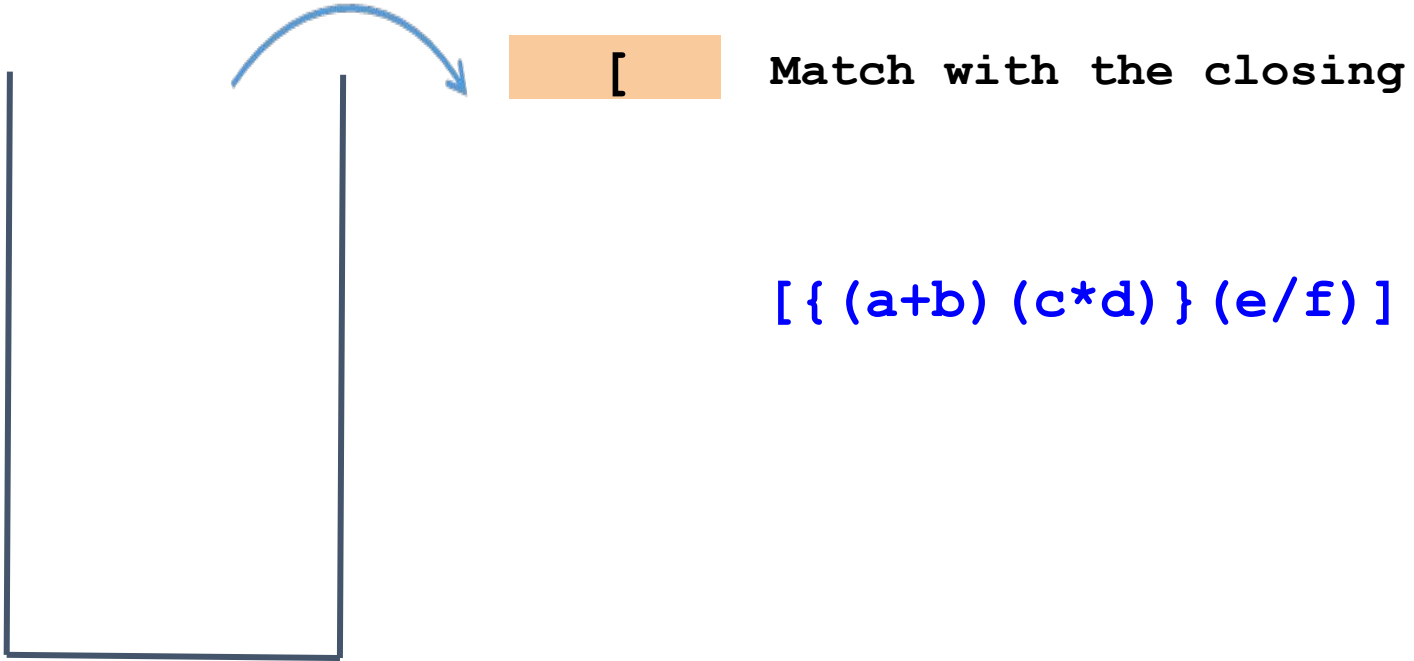


[{ (a+b) (c*d) } (e/f)]

Stack: Balanced Parentheses



[{ (a + b) (c * d) } (e / f)]



Stack: Arithmetic Expression Evaluation

(2 + 3) * 3 / (5 - 2)



Postfix notation

2 3 + 3 * 5 2 - /

Stack: Arithmetic Expression Evaluation



2 3 + 3 * 5 2 - /

Operand: push to stack

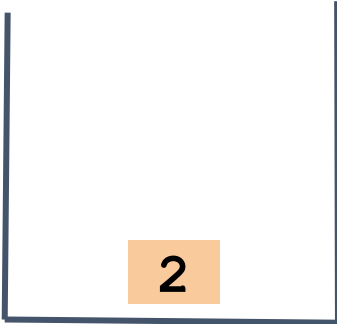


Stack: Arithmetic Expression Evaluation



2 3 + 3 * 5 2 - /

Operand: push to stack

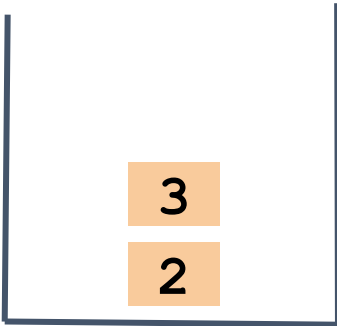


Stack: Arithmetic Expression Evaluation



2 3 + 3 * 5 2 - /

Operand: push to stack



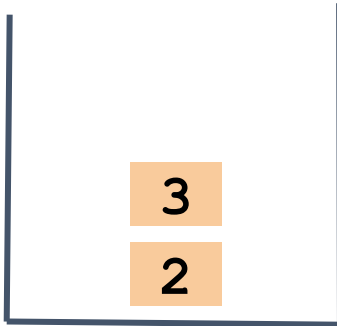
Stack: Arithmetic Expression Evaluation



2	3	+	3	*	5	2	-	/
---	---	---	---	---	---	---	---	---

Operator: pop twice and apply the operator; push the result back to the stack

$$2+3 = 5$$

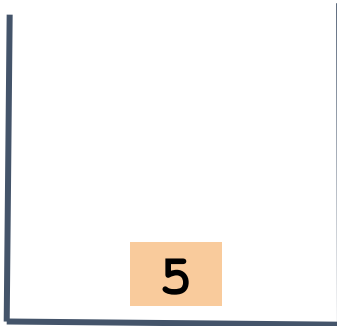


Stack: Arithmetic Expression Evaluation



2	3	+	3	*	5	2	-	/
---	---	---	---	---	---	---	---	---

Operator: pop twice and apply the operator; push the result back to the stack

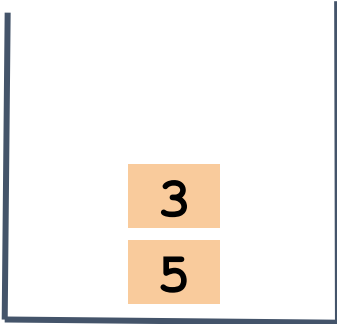


Stack: Arithmetic Expression Evaluation



2	3	+	3	*	5	2	-	/
---	---	---	---	---	---	---	---	---

Operand: push to stack



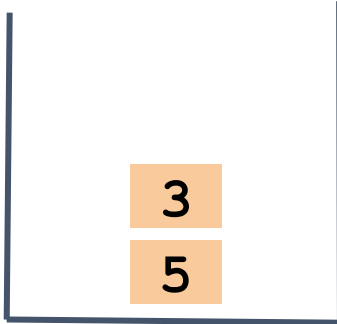
Stack: Arithmetic Expression Evaluation



2 3 + 3 * 5 2 - /

Operator: pop twice and apply the operator; push the result back to the stack

$$5 * 3 = 15$$

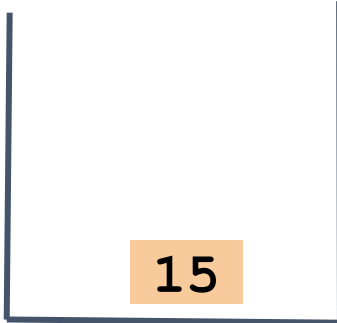


Stack: Arithmetic Expression Evaluation



2 3 + 3 * 5 2 - /

Operator: pop twice and apply the operator; push the result back to the stack

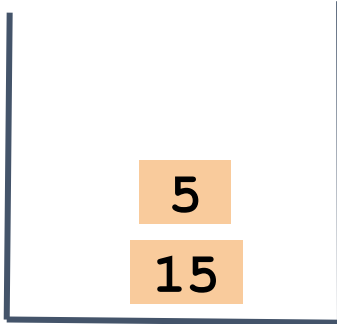


Stack: Arithmetic Expression Evaluation



2 3 + 3 * 5 2 - /

Operand: push to stack

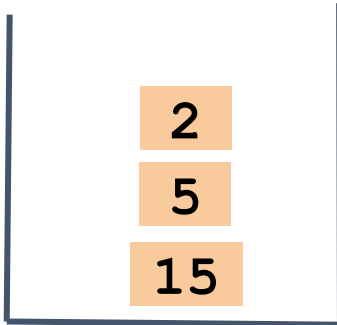


Stack: Arithmetic Expression Evaluation



2 3 + 3 * 5 2 - /

Operand: push to stack



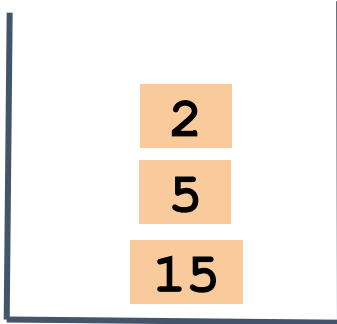
Stack: Arithmetic Expression Evaluation



2	3	+	3	*	5	2	-	/
---	---	---	---	---	---	---	---	---

Operator: pop twice and apply the operator; push the result back to the stack

$$5 - 2 = 3$$

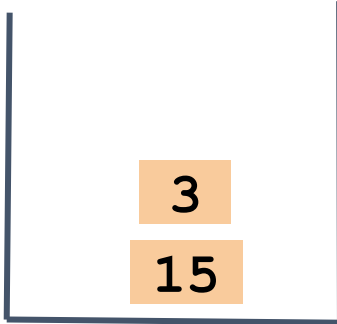


Stack: Arithmetic Expression Evaluation



2	3	+	3	*	5	2	-	/
---	---	---	---	---	---	---	---	---

Operator: pop twice and apply the operator; push the result back to the stack



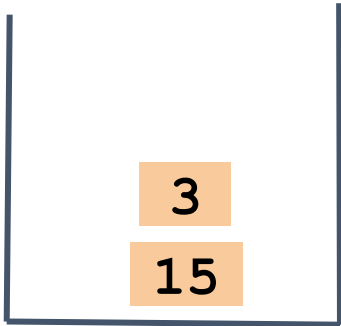
Stack: Arithmetic Expression Evaluation



2	3	+	3	*	5	2	-	/
---	---	---	---	---	---	---	---	---

Operator: pop twice and apply the operator; push the result back to the stack

$$15/3 = 5$$

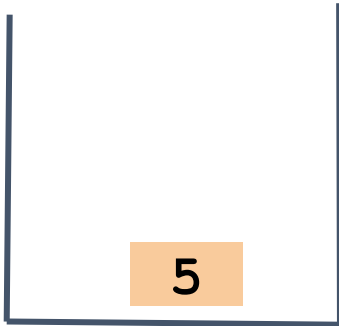


Stack: Arithmetic Expression Evaluation



2	3	+	3	*	5	2	-	/
---	---	---	---	---	---	---	---	---

Operator: pop twice and apply the operator; push the result back to the stack



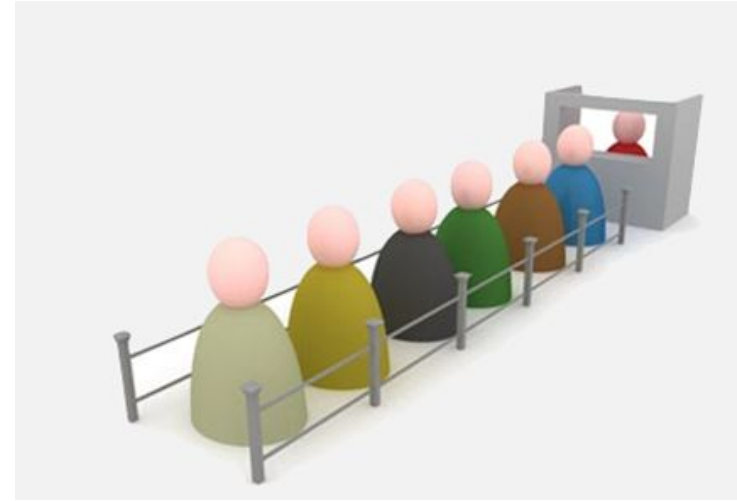
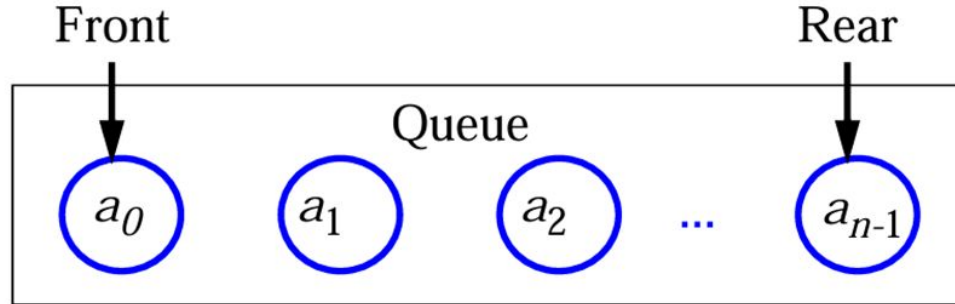
Queue

- A queue differs from a stack in that its insertion and removal processes follows the **first-in-first-out (FIFO)** principle
- Elements may be **inserted at any time**, but only the element which has been in the queue the **longest** may be **removed**



Queue

- Elements are inserted at the *rear* (**enqueued**) and removed from the *front* (**dequeued**)



Queue

- The queue supports two fundamental methods:
 - `enqueue(o)`: Insert object *o* at the rear of the queue
Input: Object; *Output*: none
 - `dequeue()`: Remove the object from the front of the queue and return it; an error occurs if the queue is empty
Input: none; *Output*: Object



Queue

- `size()`: Return the number of objects in the queue
Input: none; Output: integer
- `isEmpty()`: Return a boolean value that indicates whether the queue is empty
Input: none; Output: boolean
- `front()`: Return, but do not remove, the front object in the queue; an error occurs if the queue is empty
Input: none; Output: Object

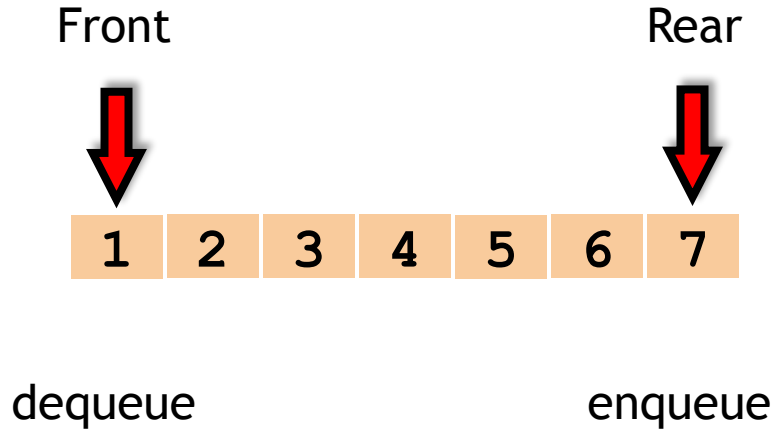


Queue

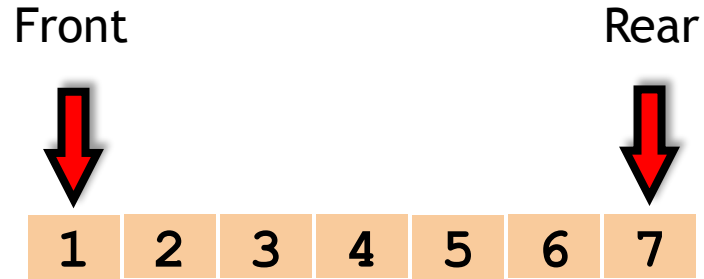
Implementing Queue is little harder than Stack.

Do you see why?

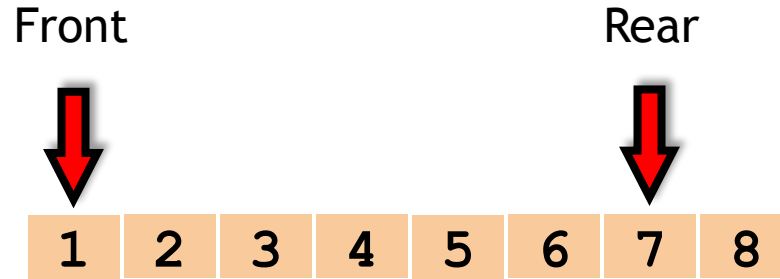
Queue Implementation



Queue Implementation: Enqueue



Queue Implementation: Enqueue



Queue Implementation: Enqueue

Front



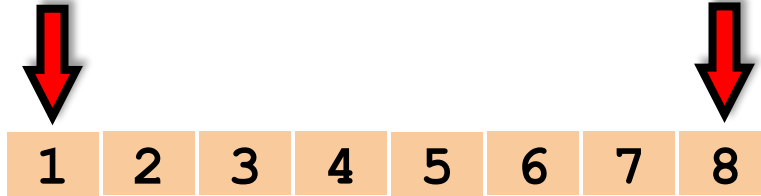
Rear



Queue Implementation: Dequeue

Front

Rear



Queue Implementation: Dequeue

